

# Trim Phase 9 — PLN, Special Expenses, Budget Pace, Monthly History, Encryption at Rest

**Date:** 2026-07-17 · **Status:** Approved by Alex (design conversation, this date) · **Builds as:** BUILD\_PLAN.md Phase 9, one task per session.

## 1. Goals

Five user-facing improvements, designed together because they share schema and math:

1. **PLN** — add Polish zloty as a display currency.
2. **Special expenses (opt-in)** — mark an expense (gift, trip) as “special”: tracked, but excluded from the monthly-budget math.
3. **Budget pace** — “given my budget and today’s date, what *should* I have spent by now?”, shown alongside “Can I afford this?”.
4. **Monthly history** — a per-month table of past spending on Analytics.
5. **Encryption at rest** — Alex (the operator) can no longer casually read users’ financial data in the Supabase dashboard.

## 2. Decisions locked with Alex (2026-07-17)

| Decision                           | Choice                                                                                                        | Why                                                                                                                                                               |
|------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Privacy depth                      | Server-side encryption of the sensitive lot (amounts, descriptions, notes, goal names, chat) — <b>not</b> E2E | E2E would break Ask Trim, the NL parser, subscription detection, and future bank sync. Honest framing: protects against casual viewing; Alex still holds the key. |
| Special-expense shape              | Simple boolean flag, no named buckets                                                                         | Least UI, fits 3-tap rule; the note field covers “what was it”.                                                                                                   |
| Special-expense exclusions         | <b>Budget math only</b>                                                                                       | Excluded from budget bars/alerts, pace, affordability, projection, wins. Still in hero cash flow, transaction list, analytics. Cash-flow numbers stay honest.     |
| Special expenses are <b>opt-in</b> | Settings toggle, <b>off by default</b>                                                                        | Alex: “don’t force the feature on anyone” — some users want gifts inside their normal budget.                                                                     |
| History placement                  | Section on the existing Analytics page                                                                        | No new nav item; analytics API already builds a monthly series.                                                                                                   |
| Category-name encryption           | Included                                                                                                      | Custom category names can be personal (“Therapy”).                                                                                                                |

### 3. Feature designs

#### 3.1 PLN currency (task 9.1 — small)

- **Migration 010\_pln\_currency.sql:** ALTER TYPE public.currency\_code ADD VALUE IF NOT EXISTS 'PLN'; (Note: ADD VALUE cannot run inside the same transaction that uses the value; run it as its own migration, nothing else in the file.)
- **Server:**
  - server/routes/me.js:9 — add 'PLN' to the Zod enum.
  - server/lib/parser.js — add 'PLN' to the enum (line 9) and to the prompt: currency list (line 26), cue rules (~line 43): "zł", "złoty", "złotych", "pln" -> PLN. PLN has grosz minor units (×100, like pence) — the “minor units” instruction already covers it; add one example ("50 zł" -> 5000).
- **Client:**
  - client/src/lib/format.js — CURRENCY\_LOCALE.PLN = 'pl-PL' (renders 50,00 zł). Fraction digits: default 2 is correct for PLN.
  - client/src/pages/Settings.jsx — add { code: 'PLN', label: 'PLN — Polish złoty (zł)' } to the CURRENCIES array.
- **No FX** — unchanged single-currency-per-user model.

#### 3.2 Special expenses (task 9.2)

**Schema — migration 011\_special\_expenses.sql:**

```
alter table public.transactions
  add column is_special boolean not null default false;
alter table public.user_stats
  add column special_expenses_enabled boolean not null default false;
```

**Preference plumbing:** special\_expenses\_enabled joins GET/PATCH /api/me (preferences), Settings gets a toggle: “Special expenses — track gifts, trips and one-offs outside your monthly budget” (off by default). The dormant rule: when the toggle is **off**, server math treats every transaction as normal (is\_special ignored, flags preserved), and the client hides all special UI. Re-enabling restores previously starred transactions.

**Server:** - transactions create/update schemas: isSpecial: z.boolean().optional() (reject/ignore on income-type transactions — expenses only). - Exclusion rule, applied **only when the user's special\_expenses\_enabled is true** — each route below already fetches or can cheaply fetch user\_stats: - routes/budgets.js — per-category spent excludes special. - routes/dashboard.js — the by-category card data (donut, top-5, “Budgets to watch” ≥75%) excludes special; **hero month in/out totals include everything** (cash flow honest); payload gains specialThisMonth (sum of this month's special expenses) and echoes the pref. - routes/projections.js — spendSoFar, amounts, last-month spend exclude special. - routes/affordability.js — spent-so-far numbers behind categoryRemaining/totalRemaining exclude special. - routes/wins.js — under-budget win detection excludes special. - routes/analytics.js — expenses **includes** special (cash flow); each series bucket gains special (that month's special total) so history can show it. - lib/askContext.js — transactions in the bundle carry is\_special; bundle notes whether the feature is enabled, so Ask Trim can answer “how much did I spend on the trip?”. - lib/subscriptions.js — unchanged (a recurring charge is a subscription regardless of the flag). - **Gamification unchanged:**

logging a special expense still fires `applyLogEvent` (streak/XP) — the user still logged.

**Client (all special UI hidden unless the preference is on):** - QuickAdd: inside the existing “Add a note or change the date” progressive-disclosure area, a “Special expense” toggle (expense mode only). Hint text: *“kept out of your monthly budget”*. Golden 3-tap path untouched. - Transactions page: star marker on special rows; one-tap star/unstar action per row (the retroactive “exclude it” button Alex asked for); same toggle in the edit dialog; a “Special” filter chip. - Dashboard hero: a compact “Special £X” chip beside In/Out, rendered only when  $> 0$ . - Settings: the enable toggle.

### 3.3 Budget pace (task 9.3)

**Server — extend GET `/api/projections/month`:** new pace object, returned **independently of the cold-start guard** (it’s plain arithmetic, valid from day 1; keep it null when the user has no monthly budget source):

```
pace: {
  target: round2(monthlyBudget * daysElapsed / daysInMonth),
  spent:  round2(spendSoFarExcludingSpecial),
  delta:  round2(target - spent),    // positive = under pace
}
```

`monthlyBudget` = sum of monthly budgets, **or `user_stats.monthly_limit` when `simple_mode` is true** (projections must start reading `user_stats`; today it doesn’t). `pace`: null when neither exists.

**Client:** - Normal mode: a pace line rendered with the “Can I afford this?” card (Alex’s requested placement): *“By day 17, about £425 of your £750 budget would typically be used — you’re at £389.”* Under/on pace -> quiet emerald tick; over -> amber, friendly copy (“a touch ahead of pace — plenty of month left”). Never red (tone rule). - Simple mode: same line inside `SimpleMonthCard` against `monthly_limit`. - Hidden when pace is null. Reuses the existing `['projections', 'month']` query — no new fetch.

### 3.4 Monthly history (task 9.4)

**Server:** - `routes/analytics.js`: series buckets gain special (see 3.2). No new endpoint — the route already accepts `?months=` up to 24. - `routes/transactions.js`: optional `?month=YYYY-MM` query param -> SQL date-range filter (`gte/lt` on month bounds), so months older than the 200-row window load correctly. Zod-validate the param shape.

**Client:** - Analytics page fetches `months=24` once; the existing 6-month chart slices the last 6; a new **“Monthly history”** section renders the rest: one row per month, newest first — **Spent / Income / Net / Special** (Special column only when the pref is on and any value  $\neq 0$ ). Trim leading months with no data (before signup). Current month included, labelled “so far”. - Each row links to `/transactions?month=YYYY-MM`; the Transactions page reads the param via `useSearchParams` to seed `monthFilter` **and** passes it to the API fetch (query key `['transactions', 'month']`).

### 3.5 Encryption at rest (task 9.5 — build LAST; riskiest, touches every money route)

**Verified 2026-07-17 (do not re-derive):** all money/text computation happens in JS on the Express server after fetching rows — zero `.rpc()` calls, zero SQL aggregates, zero `ORDER BY`

amount anywhere in server/routes/ + server/lib/. Exactly **two** SQL queries read a sensitive column's content, both in the merchant-suggest endpoint: routes/categories.js:77 (.ilike('description', ...)) and routes/categories.js:100 (.eq('name', keyword-Name) on categories). Both move to JS matching over decrypted rows (fetch the user's recent ~300 transactions / their category list, match in code; behaviour identical to the user).

**Crypto design — new server/lib/crypto.js (pure Node, no extensions):** - AES-256-GCM via node:crypto. Master key: new env DATA\_ENCRYPTION\_KEY (32 bytes, base64). - Per-user key: HKDF(masterKey, salt: 'trim-v1', info: userId) — ciphertext is bound to its user; a value copied into another user's row will not decrypt. - Wire format (stored in text columns): v1:<iv\_b64>:<tag\_b64>:<ct\_b64>. The v1 prefix enables future key rotation. - API: encryptField(userId, plaintext) -> string, decryptField(userId, stored) -> string; helpers encryptAmount/decryptAmount wrap Number conversion. Decrypt failures throw (500) — never silently return ciphertext. - **Fail closed, verified by review 2026-07-17:** pass { authTagLength: 16 } to createDecipheriv and assert the decoded tag is exactly 16 bytes — Node otherwise accepts a 4-byte tag with only a deprecation warning, cutting forgery cost from  $2^{128}$  to  $2^{32}$  for an attacker who can write to the database (the exact threat this encryption addresses). decryptAmount must throw unless the result Number.isFinite — Number('') is 0 and Number('abc') is NaN, so a silent empty amount would become a zero-value transaction and NaN would poison every total. - dev:mock (scripts/devMock.js) is untouched — it's in-memory plaintext by design.

**Encrypted columns** (each becomes a text \_enc column; plaintext column dropped in the final step):

| Table                  | Columns                             |
|------------------------|-------------------------------------|
| transactions           | amount, description                 |
| budgets                | amount_limit                        |
| categories             | name                                |
| savings_goals          | name, target_amount, current_amount |
| savings_contributions  | amount, note                        |
| subscription_overrides | display_name                        |
| user_stats             | monthly_limit                       |
| ask_messages           | content                             |

**Stays plaintext (needed for queries; reveals no spending content):** dates, type, ids/category\_id, icon/color/sort\_order, is\_special, streak/XP/level, simple\_mode, special\_expenses\_enabled, currency, display\_name (identity, not finances — Supabase Auth shows emails regardless).

**Migration choreography (3 steps, in one session, verified between):** 1. 012\_encryption\_columns.sql — add nullable \*\_enc text columns alongside existing ones. 2. server/scripts/encrypt-backfill.mjs — for every row: encrypt plaintext -> \_enc, write, then **re-SELECT the row and decrypt what the database actually returned**, comparing against the original; abort loudly on any mismatch. Idempotent. Requires DATA\_ENCRYPTION\_KEY + service key. Print row counts per table.

**Two defects were found by review in the first implementation of this script (2026-07-17) — do not reintroduce them:** - **Paging must not rely on the write shrinking the result set.** Filtering on .is(first\_enc, null) and expecting the write to remove the row loops forever on any row

whose plaintext is NULL (it gets written as NULL and re-matches). `user_stats.monthly_limit` and `subscription_overrides.display_name` are both nullable, so this hangs mid-run against production. Use keyset pagination (order by PK, `.gt(pk, cursor)`) and keep the `is null` filter only for idempotency. - **The verification must read back from the database.** Comparing `decryptField(justEncryptedValue)` to the original is only `decrypt(encrypt(x)) === x` — a unit test, blind to truncating columns, encoding mangles, or writes that never landed. Since migration 013 irreversibly drops the plaintext on the strength of this check, it must re-read. Also verify **after** the write, and never let a failed row be skipped by the idempotency filter on re-run. 3. `013_encryption_drop_plaintext.sql` — run **only after** the backfill verifies and the app has been click-tested reading/writing encrypted data. Drops plaintext columns and renames `*_enc` -> original names (so route code refers to one name). - Numeric Zod validation (positive, finite,  $\leq 1e9$ ) stays at the API boundary — unchanged.

**Route changes:** the nine amount-touching routes (affordability, analytics, budgets, dashboard, goals, projections, subscriptions, transactions, wins)+categories, ask, me switch to encrypt-on-write / decrypt-after-fetch via the helpers. Mechanical but wide — this is why it's the last task, after the feature math above has settled.

**Category-seeding trigger:** the `handle_new_user` SQL trigger seeds 12 default categories by plaintext name; Postgres never holds the key, so it cannot encrypt. Seeding moves to the server: on GET `/api/me` with zero categories, insert the 12 defaults through the API path with encrypted names (mirrors the existing lazy `user_stats` insert). The trigger keeps only its `user_stats` part.

**Honest limits — copy into SECURITY.md verbatim:** - The server (and therefore Alex, who deploys it) holds `DATA_ENCRYPTION_KEY`. This protects against **casual or accidental viewing** (Supabase dashboard, SQL console, DB backups show ciphertext) and supports a truthful “your financial data is encrypted at rest” statement. It is **not** protection against a malicious operator. True E2E was considered and rejected (kills Ask Trim / parser / subscription detection / bank sync). - Ask Trim still sends decrypted context to Anthropic per question — unchanged by this work. - **Key loss = permanent loss of all users' financial data.** Alex must back up `DATA_ENCRYPTION_KEY` outside the hosting dashboard (e.g. `~/Keys/`, same place as the Enable Banking key). Set it in Vercel env + `server/.env`.

#### 4. Build order (BUILD\_PLAN Phase 9)

| Task | Scope                                              | Why this order                                        |
|------|----------------------------------------------------|-------------------------------------------------------|
| 9.1  | PLN                                                | Tiny, independent.                                    |
| 9.2  | Special expenses (schema + pref + route math + UI) | Feature math others depend on.                        |
| 9.3  | Budget pace                                        | Rides on projections + 9.2's exclusions.              |
| 9.4  | Monthly history                                    | Rides on analytics + 9.2's special buckets.           |
| 9.5  | Encryption at rest                                 | Touches every route; land after feature math settles. |

**Migration numbering collision:** the (unbuilt) Phase 8 spec reserved DDL numbers 010/011. Phase 9 ships first and takes **010–013**; Phase 8's DDL becomes **014+** when built. A note goes

into BUILD\_PLAN.md Phase 8 in task 9.1's session.

## 5. Definition of done (every task)

Per CLAUDE.md: `npm run build` passes in `client/`; server starts cleanly; the feature is **clicked to in the running UI** (dev server); FEATURES.md + BUILD\_PLAN.md updated (SECURITY.md in 9.5); committed, with an explicit note that the worktree branch still needs merging to `main`.

## 6. Out of scope (explicitly)

- FX conversion between currencies (unchanged rule).
- Named special-expense buckets ("Rome trip" pots) — revisit only if the flag proves insufficient.
- Historical budget snapshots (history compares months to each other, not to the budgets that existed at the time).
- Encrypting `display_name`, dates, or category ids (metadata needed for queries; see 3.5).
- Any Phase 8 (bank sync / Stripe billing) work.